

Wstęp do przetwarzania transakcyjnego

Introduction to transactional processing

dr hab. inż. Maciej Grzenda

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska

Wydział Matematyki i Nauk Informacyjnych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Challenges related to placing data in databases

- What if we save a new order in a database, which requires multiple SQL statements to be executed?
 - As an example, **one SQL INSERT statement inserting an order** record and **five SQL INSERT statements** inserting order details records may be needed
 - What if some of these statements fail?
 - How to avoid such partial changes in a database?
- Could some changes be cached in the memory of a DBMS for better performance? If so, could we lose them due to a power failure?
- What if different users submit conflicting changes to a database, e.g.:
 - User A: `DELETE FROM Customers WHERE CustomerId=1078`
 - User B: `INSERT INTO Orders (OrderId, CustomerId, ...)`
`VALUES (145, 1078, ...)`

The answer: To address all these and other challenges, **transactional processing** is used. It is fundamental to obtain high-quality data.

Objectives

- SQL is used not only for querying the data, but also for:
 - Creating new records
 - Modifying existing records
 - Deleting records
- One action in real life, such as saving a single order, may require many SQL statements
- Hence, a method is needed to avoid partial updates. Also, careful management of concurrent changes is required.
- This is why transactional processing is mandatory in database management systems used to manage core business data.

Transactional processing

- Consider an example – a batch of SQL statements needed to transfer money between bank accounts:

```
UPDATE Accounts set balance=balance-100 where  
account_id=456
```

```
UPDATE Accounts set balance=balance+100 where  
account_id=8899
```



Possible problems

- Either both UPDATE statements should be successful or none of them
- The results of statements should be persistent (*trwałe*). What would happen if they were subject to power failure?
- One batch of commands should not directly affect other statements possibly executed in parallel

Transactional processing

Data processing can be referred to as **transactional processing** when ACID conditions are fulfilled i.e. four features are fulfilled

A	C	I	D
T O M I C I T Y	O N S I S T E N C Y	S O L A T I O N	U R A B I L I T Y

Transaction in databases

- In databases, a transaction (*transakcja*) is defined by a sequence of SQL statements that have to be executed together to get a proper result
- Examples include:
 - saving one order - which may require many records to be created
 - transferring money - which may require many account records to be updated and one transfer record to be inserted

Transactions: ACID features

Feature	Description
Atomicity <i>Atomowość</i>	A transaction defined by a sequence of SQL statements must be treated as a single unit – either committed or aborted – but always as a whole. Changes arising from some statements comprising on a transaction only are not allowed.
Consistency <i>Spójność</i>	No matter what is the result of a transaction the database must remain consistent. In particular all the constraints must remain untouched.
Isolation <i>Izolacja</i>	Unfinished transactions can not affect the result of other transactions in progress. Example: Until money transfer transaction is committed no one can withdraw transferred money.
Durability <i>Trwałość</i>	The result of committed transaction must be persistent i.e. the changes to the data can not be revoked or lost.

Transaction – example

Przykład transakcji

BEGIN TRANSACTION

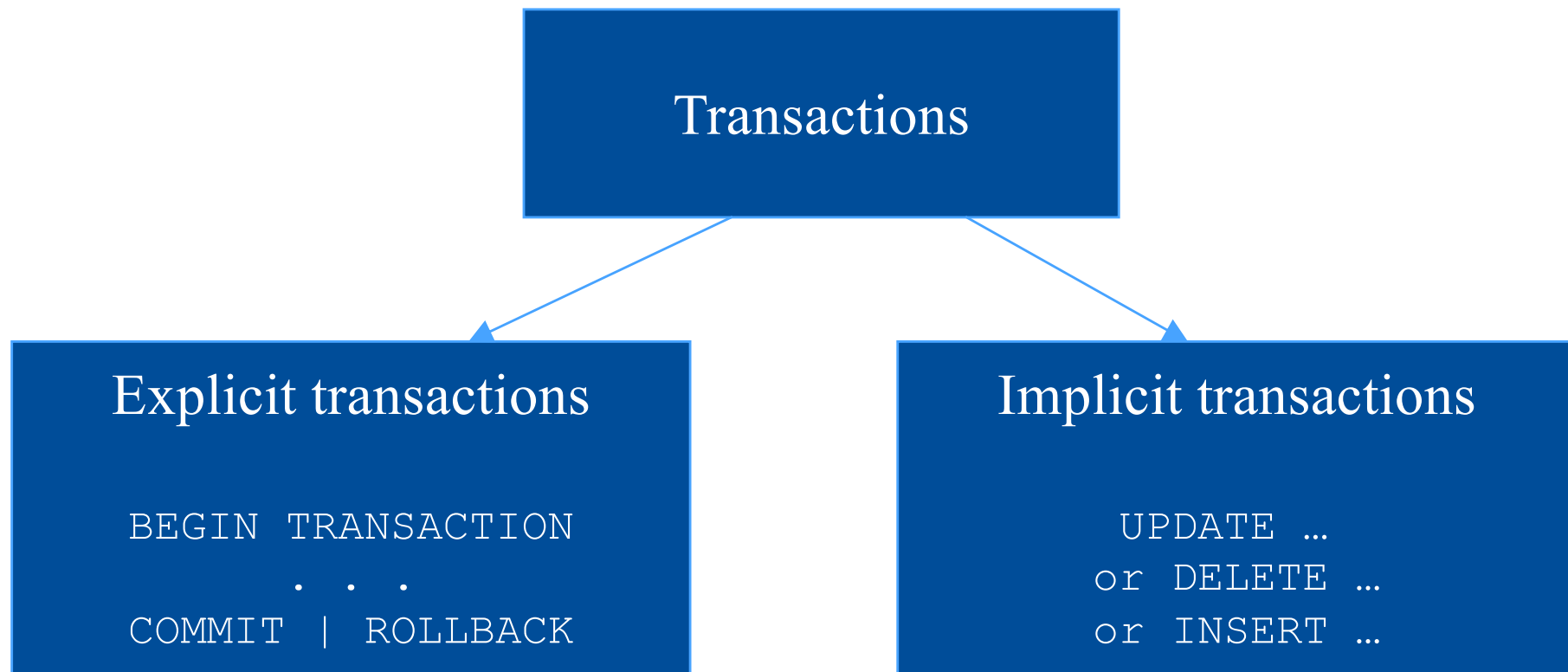
```
UPDATE Accounts set  
    balance=balance-100 where  
    account_id=456
```

```
UPDATE Accounts set  
    balance=balance+100 where  
    account_id=8899
```

COMMIT

By using **BEGIN TRANSACTION** and **COMMIT** we denote the scope of a transaction i.e. the sequence of SQL statements that should be processed in compliance with ACID features.

Transaction categories: Ms SQL Server



Implicit transactions are single statements that are guaranteed to be processed as transactions e.g. either all the rows matching WHERE or none of them will be affected by a single UPDATE statement

Transaction commands: Ms SQL Server

- BEGIN TRANSACTION – to start a transaction
- COMMIT – to commit all the changes
- ROLLBACK – to undo all the changes since the beginning of a current transaction
- Some RDBMS allow to use:
 - *nested transactions (trans. zagnieżdżone)* – transactions within transactions
 - *save points* – points in longer batches of commands that can be used to undo changes but only of a part of transaction
- Note that some DBMS platforms, such as Oracle Database will assume that the end of the previous transaction (COMMIT or ROLLBACK) means the beginning of the next one. In such cases, BEGIN TRANSACTION is not used

Transactions - discussion

- Transactional processing affects all data modification statements i.e. all INSERT, UPDATE and DELETE statements
- ROLLBACK can be used to cancel a transaction and resign from applying all the statements of a transaction
- DDL statements can be NOT a part of a transaction. For instance:
 - In Ms SQL Server, ROLLBACK will remove a table created within a transaction
 - In Oracle Database, ROLLBACK will not remove a table created within a transaction

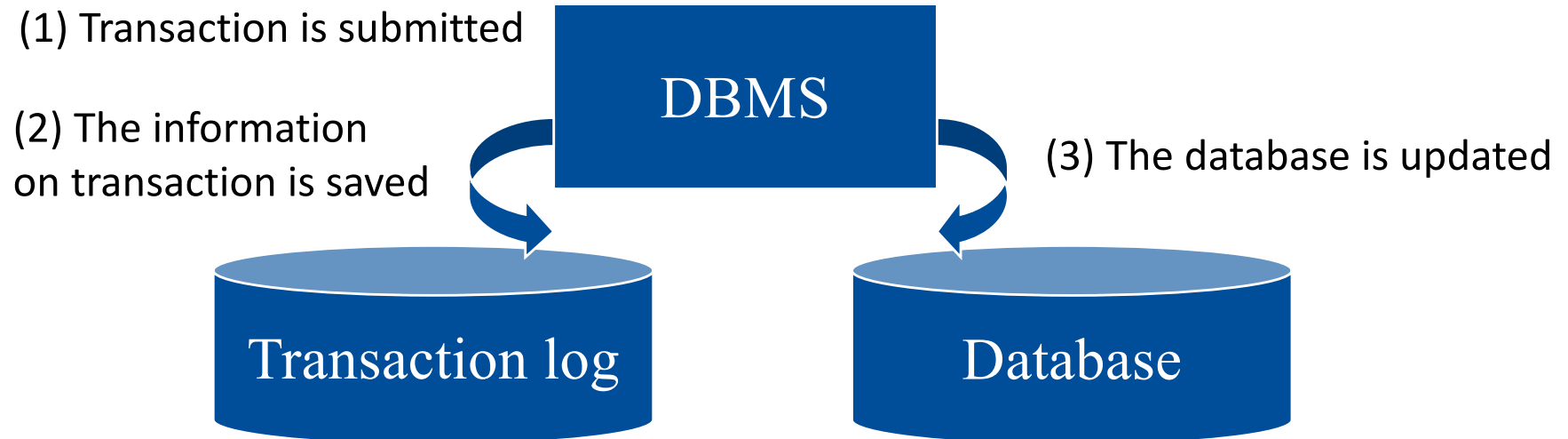
In practice, DDL statements should not be mixed with changes in the data done within the same transaction. DDL statements are supposed to be executed on separate basis as a part of carefully planned development of data model.

The implementation of transactional processing in DBMS

No straightforward solutions – ACID must be strictly followed – consider sudden lack of power supply:

- No result of committed transaction can be lost (durability),
- Partial update is not allowed (atomicity),
- Partial results can not be used by other running transactions (isolation)

Transactions - solution



The transaction log helps to identify affected records and resolve pending transactions in case DBMS is restarted after sudden failure (when no proper shutdown was performed). Note that updates to transaction log and possible roll back of partially done transactions is automatically done by a DBMS.

Changes to a database may be cached and not applied immediately. After a sudden failure, a database may contain inconsistent data, e.g. only a part of the transaction updates applied. In such a case, the transaction log is consulted to resolve each transaction when restarting the DBMS service.

Transactions - implementation

- On average over 99% of transactions are committed
- Thus, for efficiency reasons, frequently the updates are made when submitted i.e. even before COMMIT is issued by a client application. In case a transaction is cancelled, the original data is restored
- In a multi-user environment, multiple transactions are processed at the same time. Thus transaction isolation problems may occur as isolation is against transaction throughput.

DBMS and transactions

- Transactional processing requires significant overhead and sophisticated solutions from DBMS
- Most commercial DBMSs incl. Oracle, Ms SQL Server, IBM DB2 etc. are transactional systems
- Many open source DBMS do NOT support ACID transactions or have limited support for ACID. Such systems offer limited reliability for mission-critical applications such as accounting and banking solutions

Summary

- SQL is used for querying data, but also data modifications and managing data structures
- To get one meaningful result such as saving a single order, frequently many SQL statements have to be executed
- This shows the need for transactional processing to avoid partial updates only
- Transactional processing requires significant overhead, but makes high quality data possible, even in systems used by thousands of concurrent users
- Out of ACID features, isolation turns out to be particularly important when multiple users modify data, which usually is the case

Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html





**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.